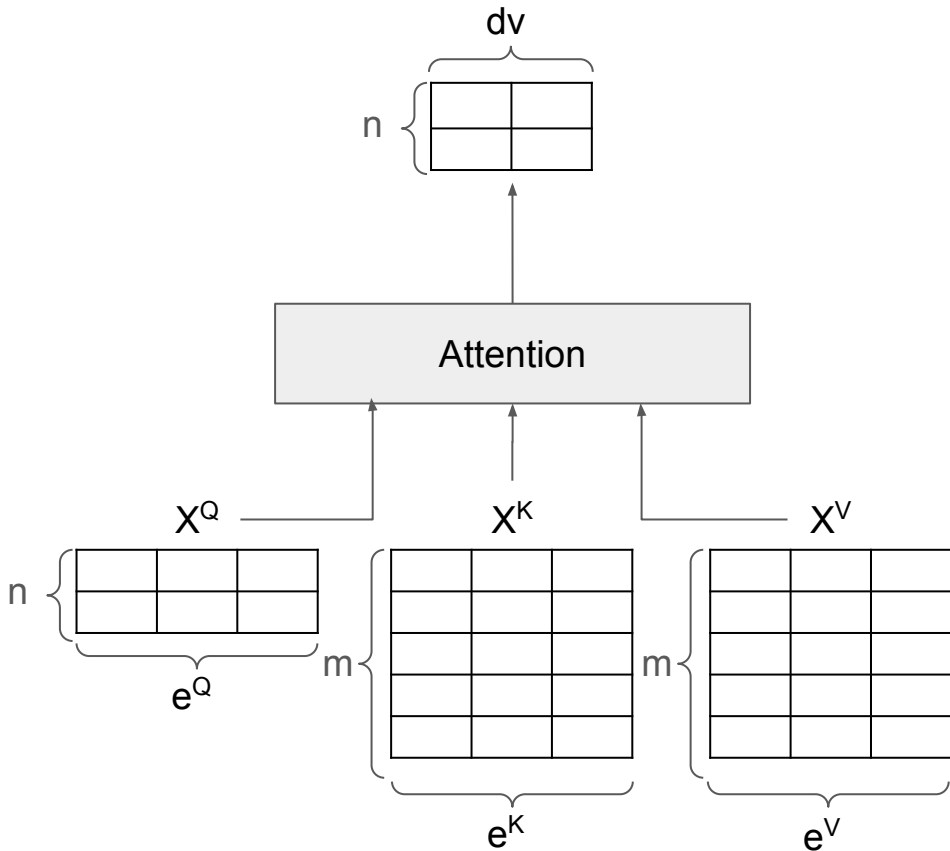


Attention

Attention is a layer that takes three 2D inputs $\{\mathbb{R}^{n \times e^q}, \mathbb{R}^{m \times e^k}, \mathbb{R}^{m \times e^v}\}$ and produces a single 2D output $\{\mathbb{R}^{n \times d^v}\}$.

As we'll see in a moment, attention resembles a weighted sum. Every query $x^Q \in X^Q$ is matched to all keys X^K . Those x^K that matched better contribute more values x^V to the result.

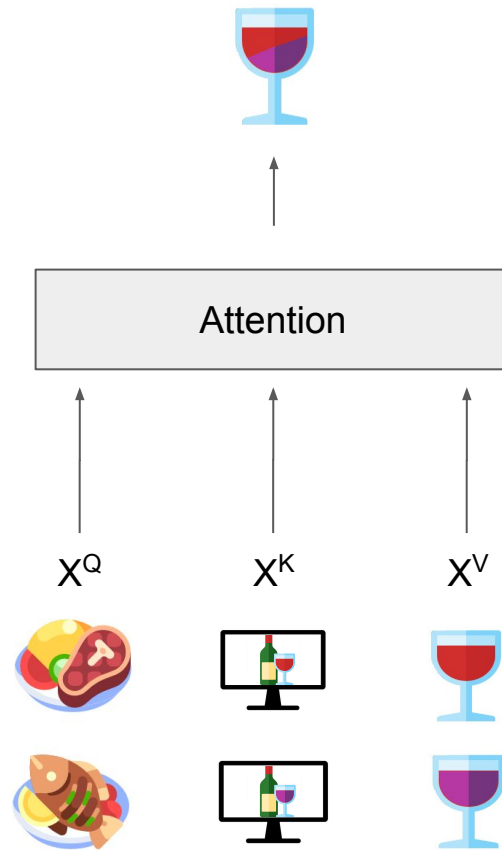
In most applications, $e^Q = e^K = e^V$.



Attention: example

Suppose we're running a restaurant. We want to serve home-made wines that match specific positions in the menu, but we only have 10 wines from Lidl.

When we're asked for wines that match some dishes, we compare the dishes' recipes X^Q to the information about the wines from the internet X^K . Then, we pour together all wines X^V , taking more of those that seemed to constitute a better match for a given dish.



Attention: implementation (dot-product attention)

Attention(X^Q, X^K, X^V) =

$$\text{softmax}\left(\frac{X^Q W^Q (X^K W^K)^T}{\sqrt{dk}}\right) X^V W^V$$

The division by $\sqrt{dk}$ makes the variance of the softmax argument equal to one, which helps with training.

n : number of query tokens

m : number of key/value tokens

e : embedding length

dk : dimension of key representation

dv : dimension of value representation

$X^Q \in \mathbb{R}^{n \times e}$: queries

$X^K \in \mathbb{R}^{m \times e}$: keys

$X^V \in \mathbb{R}^{m \times e}$: values

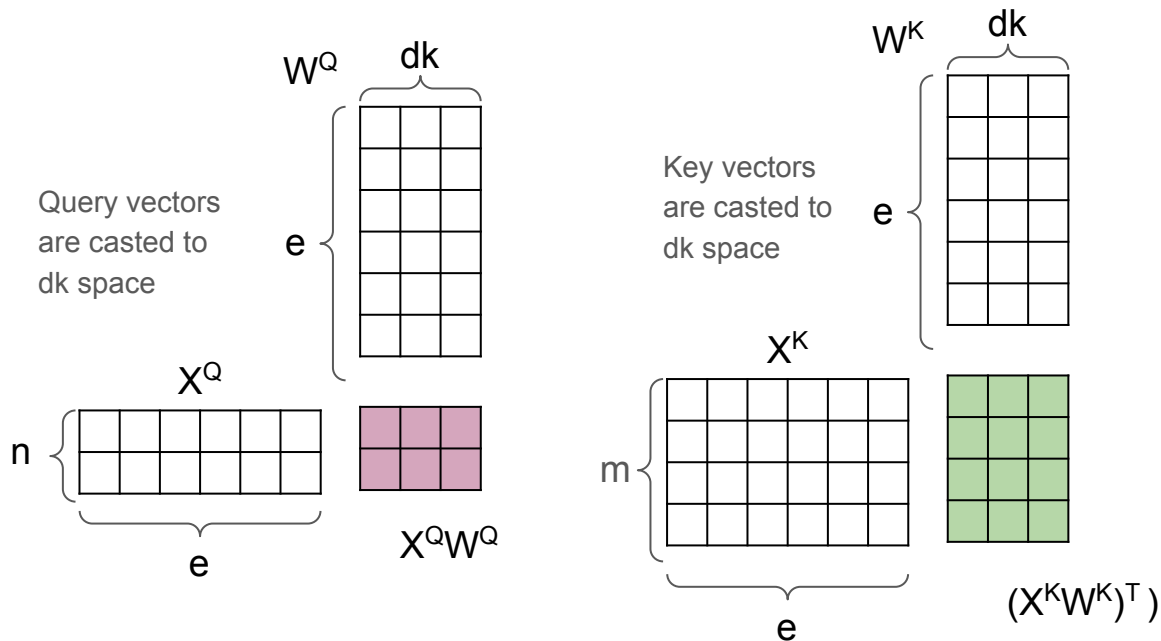
$W^Q \in \mathbb{R}^{e \times dk}$: query weights (trainable)

$W^K \in \mathbb{R}^{e \times dk}$: key weights (trainable)

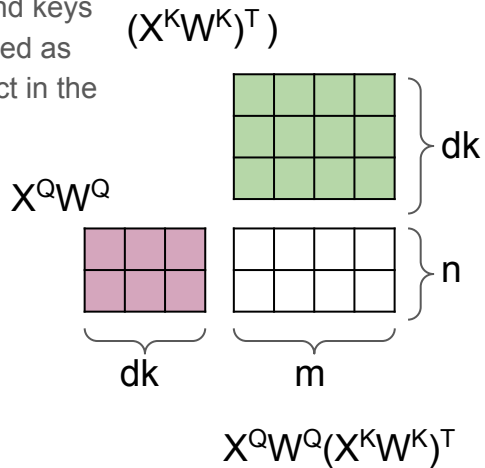
$W^V \in \mathbb{R}^{e \times dv}$: value weights (trainable)

Attention: query-key matching

$$\text{softmax}\left(\frac{X^Q W^Q (X^K W^K)^T}{\sqrt{dk}}\right) X^V W^V$$



Match between queries and keys is calculated as dot product in the dk space



Attention: are both W^Q and W^K necessary?

$$X^Q W^Q (X^K W^K)^T = X^Q W^Q (W^K)^T (X^K)^T$$

$W^Q (W^K)^T$ is a linear transformation, so instead of training both W^Q and W^K , we could use a single matrix W^{QK} .

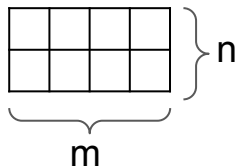
However, $W^Q \in \mathbb{R}^{e \times dk}$, $W^K \in \mathbb{R}^{e \times dk}$, so $W^{QK} \in \mathbb{R}^{e \times e}$

In the original Transformer from 2017, $e=512$, $dk=64$, so using a single square W^{QK} would lead to 4 times more parameters. The true purpose of having both W^Q and W^K is dimensionality reduction.

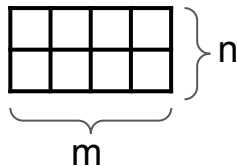
Attention: weighted sum

$$\text{softmax}\left(\frac{X^Q W^Q (X^K W^K)^T}{\sqrt{dk}}\right) X^V W^V$$

$$\frac{X^Q W^Q (X^K W^K)^T}{\sqrt{dk}}$$

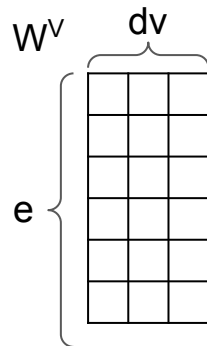
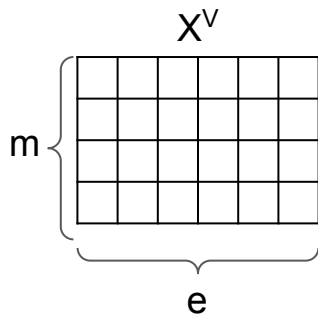


Every row sums up to one after softmax



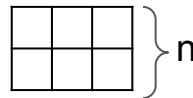
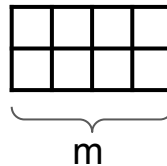
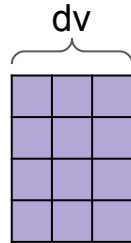
$$\text{softmax}\left(\frac{X^Q W^Q (X^K W^K)^T}{\sqrt{dk}}\right)$$

Value vectors are casted to dv space



$$X^V W^V$$

A weighted sum of $X^V W^V$ becomes the output



Attention: the order of the inputs doesn't matter

Attention(X^Q, X^K, X^V) =

$$\text{softmax}\left(\frac{X^Q W^Q (X^K W^K)^T}{\sqrt{dk}}\right) X^V W^V$$

Permutations of X^Q only change the order of the output, and permutations of (X^K, X^V) change nothing.

This is contrary to a popular intuition: since attention is used for sequences of tokens, surely it must know which token comes after which. That's not the case: if the position of a token is important, it must be encoded in the data.

Effectively, X^Q is an unordered set, while (X^K, X^V) is an unordered set of pairs.

Multi-head attention

Runs multiple attention blocks on the same inputs and concatenates the results.

MultiheadAttention(X^Q, X^K, X^V) =

$$\text{concat}_{i \in \{1, h\}} \left(\text{softmax} \left(\frac{X_i^Q W_i^Q (X_i^K W_i^K)^T}{\sqrt{dk}} \right) X_i^V W_i^V \right) W^O$$

In popular Transformer implementations,
 $dk=dv=e/h$, $do=e$.

h : number of heads

n : number of query tokens

m : number of key/value tokens

e : embedding length

dk : dimension of key representation

dv : dimension of value repr.

do : dimension of output repr.

$X^Q \in \mathbb{R}^{n \times e}$: query tokens

$X^K \in \mathbb{R}^{m \times e}$: key tokens

$X^V \in \mathbb{R}^{m \times e}$: value tokens

$W_i^Q \in \mathbb{R}^{e \times dk}$: query weights

$W_i^K \in \mathbb{R}^{e \times dk}$: key weights

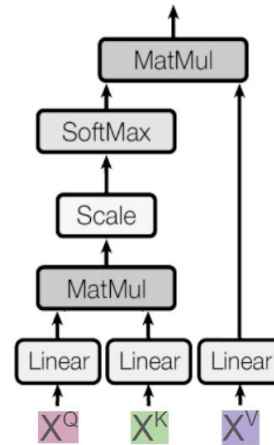
$W_i^V \in \mathbb{R}^{e \times dv}$: value weights

$W^O \in \mathbb{R}^{dv \times h \times do}$: output weights

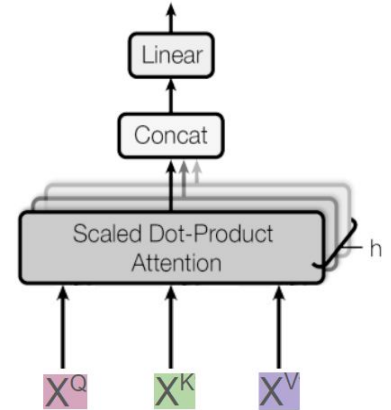
Multi-head attention diagram

$$\text{concat}_{i \in \{1, h\}} \left(\text{softmax} \left(\frac{X^Q W_i^Q (X^K W_i^K)^T}{\sqrt{dk}} \right) X^V W_i^V \right) W^O$$

Scaled Dot-Product Attention



Multi-Head Attention



Multi-head attention: are W_i^V necessary?

Similarly to W^Q and W^K , they reduce dimensionality

$$\underbrace{\text{concat}_{i \in \{1, h\}} \left(\text{softmax} \left(\frac{X^Q W_i^Q (X^K W_i^K)^T}{\sqrt{dk}} \right) X^V W_i^V \right)}_{\in \mathbb{R}^{n \times dv \times h}} W^O =$$

$$\underbrace{\text{concat}_{i \in \{1, h\}} \left(\text{softmax} \left(\frac{X^Q W_i^Q (X^K W_i^K)^T}{\sqrt{dk}} \right) X^V \right)}_{\in \mathbb{R}^{n \times e \times h}} YW^O$$

$$Y = \left\{ \begin{array}{c|ccc} & \overbrace{\hspace{1.5cm}}^{dv \times h} & & & \\ \hline W_1^V & 0 & 0 & 0 \\ \hline & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & W_2^V \\ \hline 0 & 0 & 0 & \end{array} \right\} e \times h$$

$$\begin{aligned} W_i^V &\in \mathbb{R}^{e \times dv} & W^O &\in \mathbb{R}^{dv \times h \times do} \\ Y &\in \mathbb{R}^{e \times h \times dv \times h} & YW^O &\in \mathbb{R}^{e \times h \times do} \end{aligned}$$

YW^O could be modeled as one matrix $Z \in \mathbb{R}^{e \times h \times do}$, but that would change the number of parameters from $h \cdot e \cdot dv + h \cdot dv \cdot do$ to $e \cdot h \cdot do$. In the original Transformer, that is a $h/2$ -fold increase.

Variants of attention

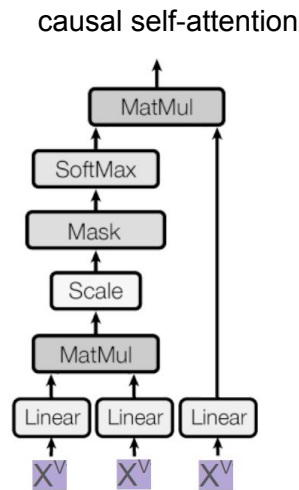
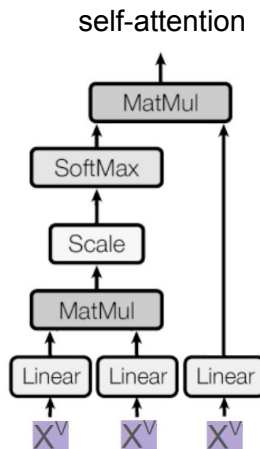
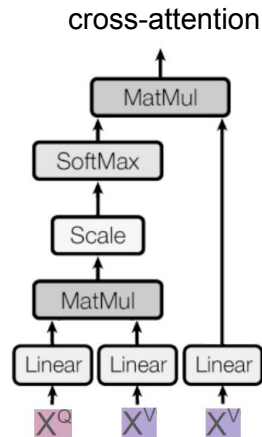
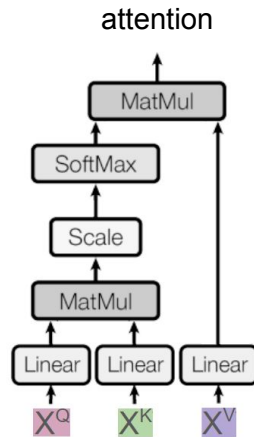
In practice, three restricted variants are mostly used.

Cross-attention: $X^Q \neq X^K = X^V$

Self-attention: $X^Q = X^K = X^V$

Causal self-attention: $X^Q = X^K = X^V$ & “earlier” tokens can’t attend to “later” ones.

Depending on the types of attention, we have three architectures: encoder-only, decoder-only and encoder-decoder.



Encoder-only Transformer

A neural network that uses self-attention, fully-connected layers, residual connections and layer normalization.

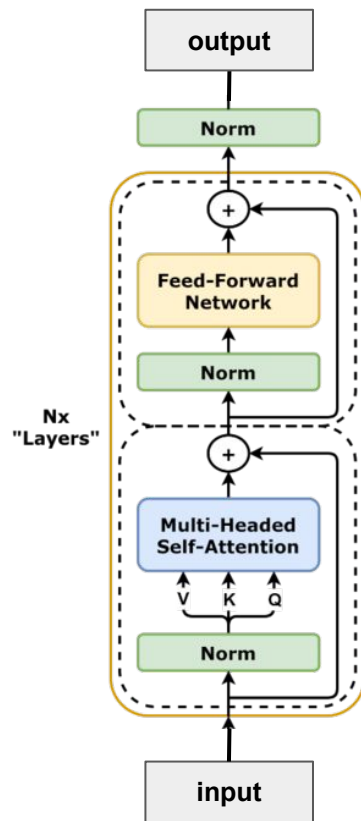
FC layers operate on individual tokens.

BERT(number of layers L /embedding size e)

BERT-Large(24/1024): 340M parameters

BERT-Base(12/768): 110M parameters

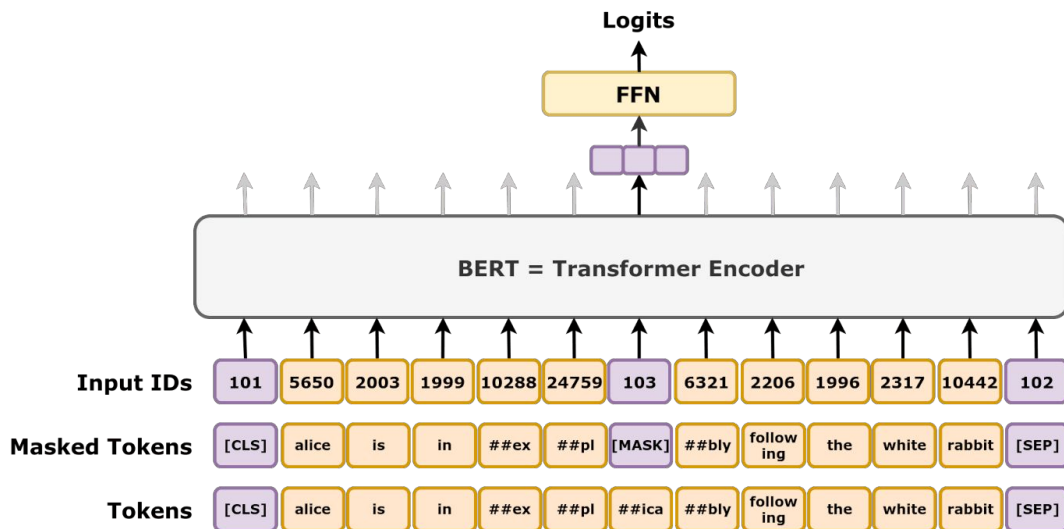
The feed-forward networks have one hidden layer of size $4 \cdot e$, and one of size e .



Pre-training encoder-only Transformers

BERT is pre-trained on a large corpus of text (3,300M words). When presented with a special tokens [MASK] in the text, the task is to predict the masked tokens and output them at given positions.

By training to do this, BERT learns about relationships in the data. How exactly it does that isn't clear.



use Adam with learning rate of $1e-4$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, L2 weight decay of 0.01, learning rate warmup over the first 10,000 steps, and linear decay of the learning rate. We use a dropout probability of 0.1 on all layers. We use a `gelu` acti-

We train with batch size of 256 sequences (256 sequences * 512 tokens = 128,000 tokens/batch) for 1,000,000 steps, which is approximately 40 epochs over the 3.3 billion word corpus.

<https://dvgodoy.github.io/dl-visuals/BERT/>

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding (2018)

Fine-tuning encoder-only Transformers

Once pre-trained on masked prediction, BERT can be fine-tuned to a different task, often on a smaller dataset. Depending on the task, small FC layers are added.

parameter	pre-train	fine-tune
lr	1e-4	3e-5
batch size	256	16, 32
epochs	40	2, 3, 4

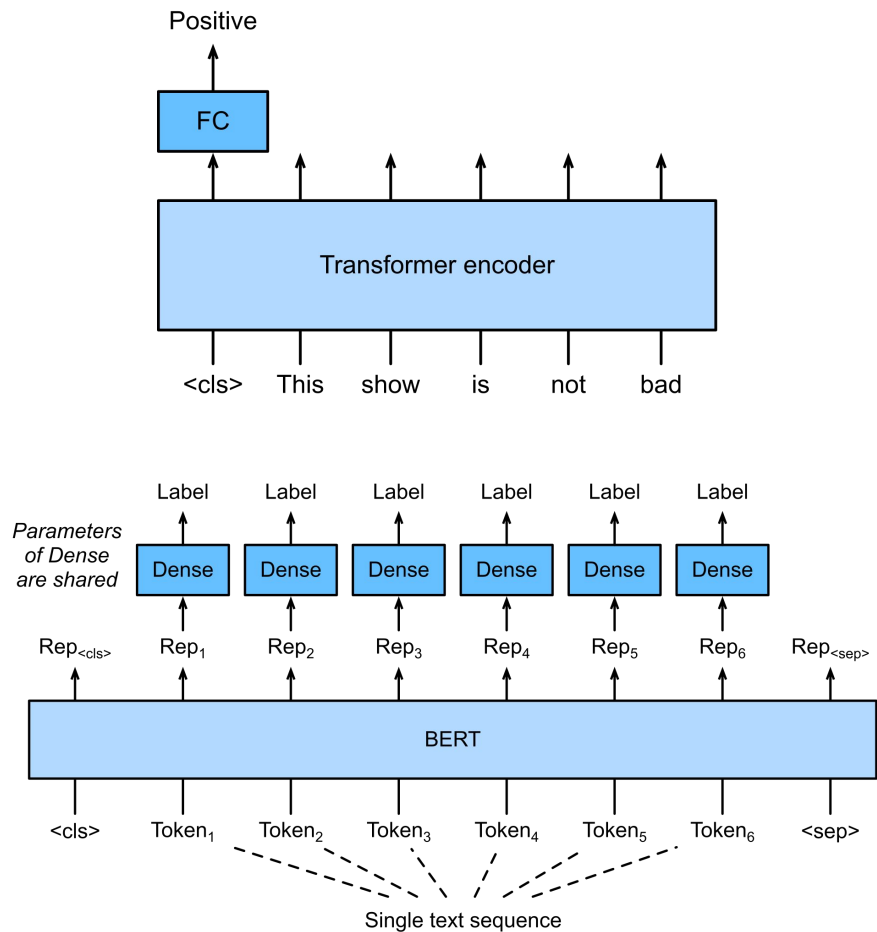
SST-2 The Stanford Sentiment Treebank is a binary single-sentence classification task consisting of sentences extracted from movie reviews with human annotations of their sentiment (Socher et al., 2013).

CoLA The Corpus of Linguistic Acceptability is a binary single-sentence classification task, where the goal is to predict whether an English sentence is linguistically “acceptable” or not (Warstadt et al., 2018).

Fine-tuning examples

Classification: instead of the whole output, classification uses only the output corresponding to the first special token <cls>.

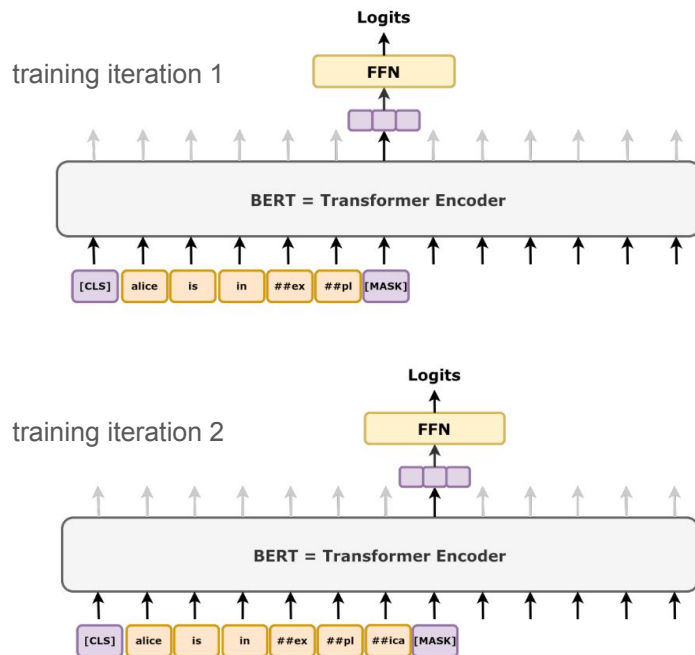
Part-of-speech tagging: each part of the output is used to predict the corresponding token's part of speech.



Encoder-only Transformers are badly fit for generation

In theory, an encoder Transformer could be used for token generation simply by appending a [MASK] token at the end. In practice, this performs poorly if the model was pre-trained on normal data because of the distribution shift.

On the other hand, pre-training only on sentences that end with [MASK] is prohibitively expensive, as only a single token could be evaluated per training iteration.



Causal self-attention

A modification to attention that makes training for generation easier.

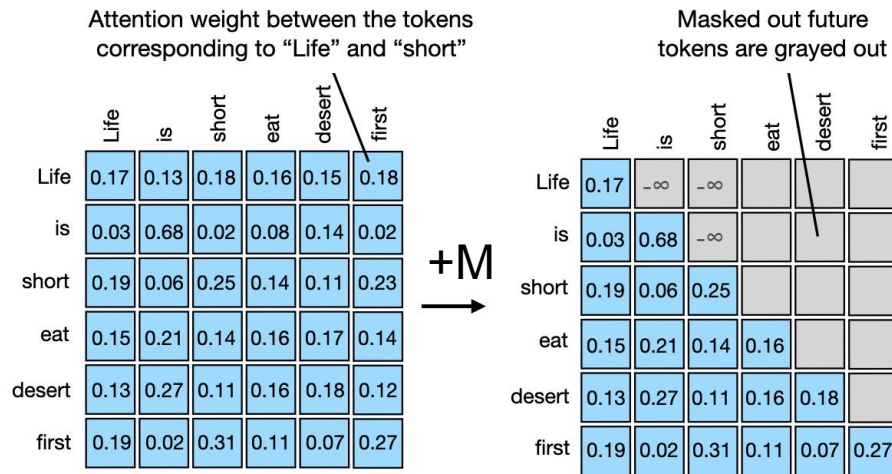
Instead of masking the inputs, softmax arguments in attention are masked: every token can attend only to itself and the previous tokens.

During training, multiple generation predictions can be evaluated at once.

Order of tokens is introduced.

Information flows in one direction.

$$\text{CausalAttention}(X^Q, X^K, X^V) = \text{softmax}\left(\frac{X^Q W^Q (X^K W^K)^T}{\sqrt{dk}} + M\right) X^V W^V$$



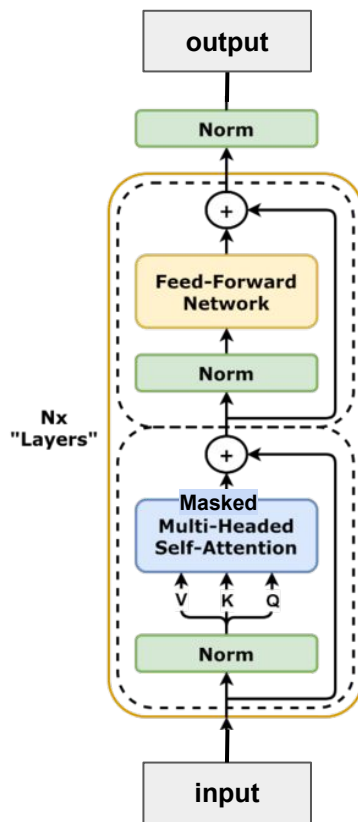
Decoder-only Transformer

Basically an encoder-only Transformer that uses causal (a.k.a. masked) self-attention.

This allows for parallel generation training, but might hurt performance on other tasks.

Llama3: 8B, 70B, 405B parameters

	8B	70B	405B
Layers	32	80	126
Model Dimension	4,096	8192	16,384
FFN Dimension	14,336	28,672	53,248
Attention Heads	32	64	128
Key/Value Heads	8	8	8
Peak Learning Rate	3×10^{-4}	1.5×10^{-4}	8×10^{-5}
Activation Function	SwiGLU		
Vocabulary Size	128,000		
Positional Embeddings	RoPE ($\theta = 500,000$)		



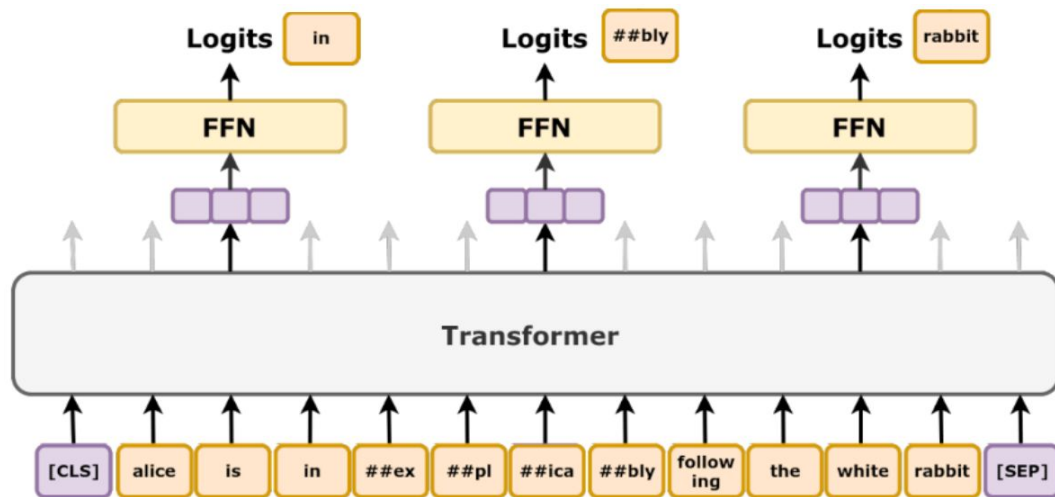
note that FC layers operate on individual tokens, so they don't require masking

Pre-training decoder-only Transformers

Next-token prediction.

Masked attention ensures every token attends only to previous tokens. No additional input masking is required.

Llama3: 15.6T tokens. 405B training took 54 days on 16K H100 GPUs. AdamW, lr 3e-4, lr schedule. Only one epoch, with an exception of Wikipedia and BookCrawl, where two epochs.



Encoder-decoder Transformer

One input is constant and analysed with an encoder, the other input is generated with a decoder. Good for seq2seq, like translation, question answering, summaries. Less popular recently, as it's harder to train, slower in chatbots, and the difference to decoder-only seems to diminish with scale.

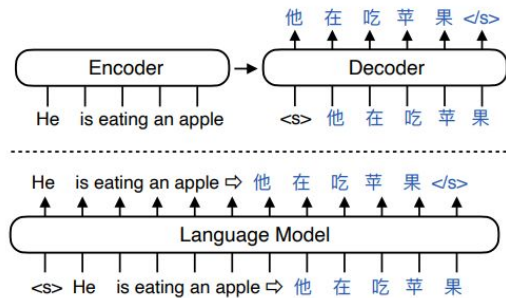
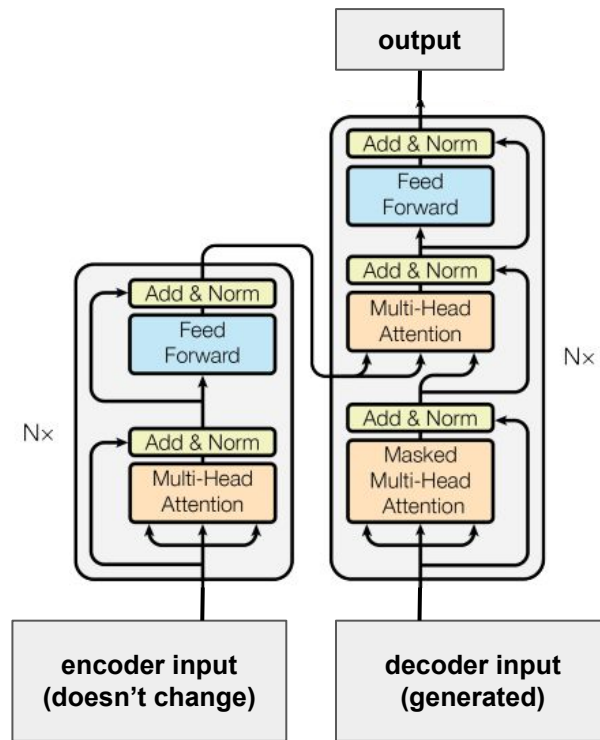


Figure 1: Encoder-Decoder (ED) framework and decoder-only Language Model (LM).



Pre-training encoder-decoder Transformers

T5 is trained by predicting removed tokens.

Data: CommonCrawl, 34B tokens.

T5: ~2.8B parameters, L=24,
e=1024, FC layer: 16384, h=32.

Can be fine-tuned to any sentence-to-sentence task, like translation, generation, image summarization etc.

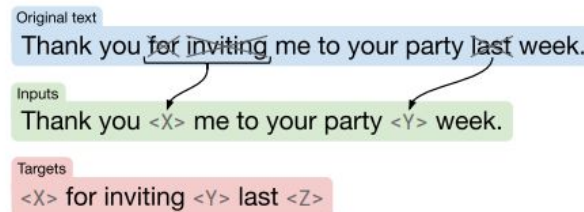


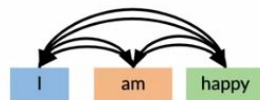
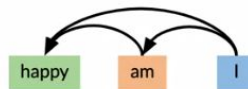
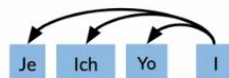
Figure 2: Schematic of the objective we use in our baseline model. In this example, we process the sentence “Thank you for inviting me to your party last week.” The words “for”, “inviting” and “last” (marked with an $\times$) are randomly chosen for corruption. Each consecutive span of corrupted tokens is replaced by a sentinel token (shown as $\langle X \rangle$ and $\langle Y \rangle$) that is unique over the example. Since “for” and “inviting” occur consecutively, they are replaced by a single sentinel $\langle X \rangle$. The output sequence then consists of the dropped-out spans, delimited by the sentinel tokens used to replace them in the input plus a final sentinel token $\langle Z \rangle$.

Transformer architectures: summary

Transformer architecture	encoder-only	decoder-only	encoder-decoder
Attention type	self-	masked self-	self-, masked self-, cross-
Popular application*	summarization	generation	translation
Examples (# parameters)	BERT (1.3B)	GPT-4 (1.8T?), Llama (405B)	T5 (2.8B), BART (406M)

Three ways of attention

- **Encoder/decoder attention:** One sentence (decoder) looks at another one (encoder)
- **Causal (self) attention:** In one sentence, words look at previous words (used for generation)
- **Bi-directional self attention:** In one sentence, words look at both previous and future words



<https://blog.sailor.plus/deep-learning/attention/>

* not exclusive: it's possible to use encoder-decoders for generation, decoders for summarization etc.

Passing text to a Transformer

Transformers operate on unordered (excl. the causal self-attention) sets of real-valued vectors. Any input must be transformed to this format first.

Text: tokenization + embedding + positional encoding. Embedding is a single trainable matrix $\mathbb{R}^{t \times e}$, where t is the vocabulary size.

universally

embeddings, despite being a single trainable matrix, already capture token relationships

Word	universally	uniformly	explicitly	widely	invariably	duly
Cosine Similarity	1.00	0.73	0.70	0.69	0.69	0.65

Tiktokenizer

It is a truth universally acknowledged, that a single man in possession of a good fortune, must be in want of a wife.

meta-llama/Meta-Llama-3-70B

Token count

26

It is a truth universally acknowledged, that a single man in possession of a good fortune, must be in want of a wife.

universally

embedding

0.1	0.3	...	0.8
-----	-----	-----	-----

+ positional encoding (5th token)

0.2	0.1	...	0.1
-----	-----	-----	-----

=

0.2	0.1	...	0.1
-----	-----	-----	-----

Transformer input

e

Passing image to a Transformer

Similar to passing text: dividing image into (overlapping) patches + embedding + positional encoding.

Embedding is a single trainable matrix $\mathbb{R}^{t \times e}$, where t is a flattened patch size. Positional encoding can include 2D information, but 1D seems to work fine.

Vision Transformer is an encoder-only Transformer trained for classification (86M parameters, later scaled to 22B).

